

JAVA

03

Web Technologies in Java

Build dynamic, server-driven web apps — from your first Servlet to a full MVC portal backed by a live database.

DURATION	LEVEL	MODULE	UPDATED
12 Sessions	Intermediate	Module 3	June 2026

IN THIS MODULE

01	Introduction to Web Architecture
02	Introduction to Servlets
03	HTTPServlet & Request Handling
04	ServletConfig & ServletContext
05	Response Handling & Redirection
06	Session Tracking
07	JSP Basics
08	JSP Actions & Tags
09	MVC Pattern Using Servlet & JSP
10	Form Validation & Error Pages
11	JDBC Integration with Servlets
12	Servlet & JSP Review

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Web Architecture

◆ Learning Objectives

- Explain the client-server model and the HTTP request-response cycle
- Distinguish between static and dynamic web pages with real examples
- Describe how a web server receives, processes, and responds to requests
- Trace the data flow from a React frontend through a Java Servlet to a MySQL database
- Identify the role of each layer in a Java web application stack

01 The Client-Server Model

Every website you visit — from IRCTC to Flipkart — runs on a client-server model. Your browser (the client) sends a request; a server receives it, processes it, and sends back a response. This request-response cycle is the heartbeat of the web.

CONCEPT — HTTP Request-Response Cycle

HTTP (HyperText Transfer Protocol) is the language clients and servers use to talk. When you search for a train ticket on IRCTC, your browser sends an HTTP GET request. The server reads it, queries a database, and sends back an HTTP response containing the results page.

HTTP REQUESTS SERVED DAILY — GLOBALLY

5 Trillion+

Source: Cisco Annual Internet Report, 2022

02 Static vs Dynamic Web Pages

A static page looks the same for every visitor — like a printed flyer. A dynamic page changes based on who is viewing it, what they searched, or what is in the database. Think of how Swiggy shows you different restaurants based on your Ahmedabad pin code versus someone in Mumbai.

Static Web Page

Fixed HTML files served as-is. Fast, cheap to host.
Example: a simple company brochure page. Content changes only when a developer edits the file.

Dynamic Web Page

Content generated at runtime from a database or logic. Example: your Paytm dashboard showing live balance and transactions. Requires a server-side language like Java.

TRY THIS LIVE

Open any website (e.g., flipkart.com) in Chrome. Press F12 -> Network tab -> reload the page. Watch the list of HTTP requests fire one by one. Click any request to see its headers, status code, and response body.

03 How Web Servers Handle Requests

A web server is software that listens on a port (usually 80 or 8080) for incoming HTTP requests. In Java web development, Apache Tomcat is the most common server. When a request arrives, Tomcat looks at the URL, maps it to the correct Java Servlet, runs the code, and sends the response back to the browser.

HTTP Status Code	Meaning	When You See It
200 OK	Request succeeded	Page loads normally
301 Moved Permanently	Resource has a new URL	Old links auto-redirect
302 Found (Redirect)	Temporary redirect	Login redirects to dashboard
404 Not Found	Resource missing on server	Wrong URL typed
500 Internal Server Error	Server-side code crashed	Unhandled Java exception

04 Java Web Application Architecture

A professional Java web app has three clear layers: the View (HTML/JSP/React that the user sees), the Controller (Java Servlet that handles logic), and the Model (database that stores data). Understanding this flow is the foundation of everything in this module.

CONCEPT — Three-Layer Flow

React (View) -> sends HTTP request -> Java Servlet (Controller) -> queries -> MySQL (Model) -> result flows back up the chain to the browser. Every session in this module adds one more piece to this pipeline.

CASE STUDY — BookMyShow India

Context — BookMyShow is India's largest ticketing platform, headquartered in Mumbai, handling millions of concurrent users during IPL or movie releases.

Challenge — During a popular movie release, thousands of users hit the ticket booking page simultaneously. A static page cannot handle user-specific seat selection and payment.

What they did — BookMyShow uses a Java-based backend where each booking request hits a

Servlet/Controller, which checks seat availability in a live database and returns a personalised response to each user.

Result — The platform successfully handles over 3 million transactions per day, with dynamic pages generated per user session.

Takeaway — Dynamic web architecture — exactly what you are learning — is what separates a real product from a brochure website.

Source: BookMyShow Engineering Blog, 2023

WHERE TO GO NEXT

You now understand why static HTML is not enough — Java gives the web its brain.

Explore deeper — MDN HTTP Overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

Try on your own — Draw the full request-response cycle for IRCTC ticket booking on paper — label the client, server, database, and HTTP verbs.

Connect the dots — Next session you will write your first Java Servlet, which sits exactly in the Controller layer you learned today.

Going further — Read about HTTP/2 and HTTP/3 — how multiplexing solves the performance bottleneck of HTTP/1.1.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web architecture tutor. Explain the client-server model and HTTP request-response cycle in simple Hinglish. Use the example of a student booking a hostel room online in India. Keep it under 150 words.

SESSION

02

60 MIN · SERVLETS

Introduction to Servlets

◆ Learning Objectives

- Define what a Java Servlet is and why it replaced older CGI scripts
- Describe the complete Servlet lifecycle: `init()`, `service()`, `destroy()`
- Configure a Servlet in `web.xml` with URL mapping
- Set up Apache Tomcat and deploy a working Servlet
- Write and run a Hello Servlet that returns HTML to the browser

01 What Is a Servlet?

A Servlet is a Java class that runs on the server and responds to HTTP requests. Think of it as a security guard at a gate: every request that comes in must go through the Servlet first. It reads the request, does its job (query a database, run logic), and writes the response back.

CONCEPT — Servlet vs CGI

Before Servlets, servers used CGI scripts. CGI spawned a new process for every request — like hiring a new employee for each customer. Servlets are far more efficient: one Servlet instance handles thousands of requests using threads, like one experienced cashier managing a fast queue at a Reliance Fresh billing counter.

CGI Script

New OS process per request. Heavy on server memory. Slow for high-traffic sites. Rarely used in modern Java.

Java Servlet

Single instance, multithreaded. Memory efficient. Handles thousands of concurrent users. Foundation of Spring, Struts, and all Java web frameworks.

02 The Servlet Lifecycle

Every Servlet goes through three lifecycle stages managed by the Servlet container (Tomcat). Understanding this is critical — it tells you where to put one-time setup code, per-request logic, and cleanup code.

Lifecycle Method	Called When	What to Put Here
<code>init()</code>	Once when Servlet first loads	Database connection setup, config loading
<code>service()</code>	Every HTTP request	Routes to <code>doGet()</code> or <code>doPost()</code>

Lifecycle Method	Called When	What to Put Here
destroy()	When Tomcat shuts down	Close DB connections, release resources

CONCEPT — service() Routes to doGet / doPost

The service() method is called automatically for every request. It checks the HTTP method (GET, POST, etc.) and delegates to doGet() or doPost(). You almost never override service() directly — just override the method you need.

03 Configuring Servlets with web.xml

Tomcat needs to know which URL should trigger which Servlet. This mapping lives in web.xml inside your WEB-INF folder. You declare the Servlet class name and then map it to a URL pattern.

```
<servlet>                                Declare the Servlet
  <servlet-name>HelloServlet</servlet-name>  Internal name (you choose)
<servlet-class>com.tops.HelloServlet</servlet-class>
  Fully qualified class name
</servlet>
<servlet-mapping>                         Map Servlet to a URL
  <servlet-name>HelloServlet</servlet-name>  Must match the name above
  <url-pattern>/hello</url-pattern>          Browser visits /hello to trigger this Servlet
</servlet-mapping>
```

04 Writing Your First Servlet

```
import javax.servlet.http.*;              Import HttpServlet classes
import java.io.*;                          Import PrintWriter
public class HelloServlet extends HttpServlet {
  Extend HttpServlet
  protected void doGet(HttpServletRequest req, HttpServletResponse res)
    Handle GET request
    throws ServletException, IOException {
    res.setContentType("text/html");        Tell browser it is HTML
    PrintWriter out = res.getWriter();      Get output stream
    out.println("<h1>Hello from TOPS Servlet!</h1>");
    Write HTML response
  }
```

}

TRY THIS LIVE

Create a Dynamic Web Project in Eclipse. Add the HelloServlet class above, configure web.xml, and run on Tomcat. Visit <http://localhost:8080/yourapp/hello> in the browser. You should see 'Hello from TOPS Servlet!' displayed.

CASE STUDY — IRCTC Ticket Booking (NIC, India)

Context — IRCTC is one of India's highest-traffic web portals, running on Java-based server technology managed by NIC.

Challenge — During Tatkal booking windows, over 5 lakh users hit the booking endpoint within seconds. The system must accept, validate, check availability, and confirm tickets reliably at massive scale.

What they did — IRCTC's backend uses Java Servlet-based architecture where each booking request is handled by a Servlet that routes to business logic and database. Tomcat clusters handle the concurrent load.

Result — The portal handles millions of transactions daily. The Servlet-based architecture allows horizontal scaling by adding more Tomcat instances behind a load balancer.

Takeaway — The Servlet you write today is the same foundational technology that powers India's busiest transactional web portal.

Source: NIC India Technology Stack Documentation, 2022

WHERE TO GO NEXT

You can now write a Servlet that responds to browser requests — the gateway to all Java web development.

Explore deeper — Oracle Java EE Servlet Tutorial: <https://docs.oracle.com/javaee/7/tutorial/servlets.htm>

Try on your own — Modify your HelloServlet to display today's date and the current server time in the HTML response.

Connect the dots — Next session you will handle GET and POST parameters — reading form data that users type and submit.

Going further — Research `@WebServlet` annotation — the modern way to configure Servlets without web.xml, available since Servlet 3.0.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java web development instructor. Explain the Servlet lifecycle (init, service, destroy) in simple Hinglish using the analogy of a tea stall opening, serving customers, and closing at night. Keep it under 150 words.

SESSION

03

60 MIN · SERVLETS

HTTPServlet & Request Handling

◆ Learning Objectives

- Differentiate between HTTP GET and POST methods and when to use each
- Use `doGet()` and `doPost()` to handle different types of requests
- Read single and multi-value form parameters using `getParameter()` and `getParameterValues()`
- Build a login form that sends data to a Servlet and returns a welcome or error message
- Inspect request headers and understand their role in web communication

01 GET vs POST — The Two Main HTTP Methods

Every browser request uses an HTTP method that tells the server what the client wants to do. GET is used to retrieve data (like loading a search results page). POST is used to send data to the server (like submitting a login form or placing an order).

HTTP GET

Parameters appear in the URL:
`/search?city=Ahmedabad`. Visible, bookmarkable, and cached. Use for read-only, non-sensitive operations.
Max URL length ~2048 characters.

HTTP POST

Parameters are in the request body — not visible in the URL. Use for sensitive data (passwords, OTPs) or large payloads. Not cached or bookmarked by default.

CONCEPT — `doGet()` and `doPost()`

In `HttpServlet`, `doGet()` handles GET requests and `doPost()` handles POST requests. When a user types a URL in the browser, that is always a GET. When a user fills a form and clicks Submit, that is usually a POST. Override the correct method based on the HTML form's method attribute.

02 Reading Request Parameters

When a user submits a form, the field values are attached to the HTTP request. Inside `doPost()`, you retrieve them using `request.getParameter('fieldName')` — where `fieldName` matches the HTML input's name attribute exactly.

```
String username = request.getParameter("username");
```

Read single form field

```
String password = request.getParameter("password");
```

Read password field

```
String[] hobbies = request.getParameterValues("hobby");
```

Read multi-value fields (checkboxes)

```
Enumeration<String> names = request.getParameterNames();
```

List all parameter names in the request

WARNING — Always Null-Check Parameters

`request.getParameter()` returns null if the field name does not exist or was left blank. Always check for null before using the value: `if (username == null || username.isEmpty())` — otherwise your Servlet will throw a `NullPointerException`.

03 Reading Request Headers

Every HTTP request carries headers — key-value pairs that describe the request. The User-Agent header tells you which browser the user is on. The Content-Type header tells the server how the request body is formatted. You can read these in a Servlet using `getHeader()`.

04 Building a Login Form + Servlet

The classic hands-on demo: an HTML login form that sends username and password to a Servlet. The Servlet reads the values, checks them, and prints a welcome message or an error — exactly how every login page on the web works at its core.

```
<form action="LoginServlet" method="post">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

POST to LoginServlet URL

Name must match `getParameter()` key

Password field

TRY THIS LIVE

Create `login.html` with the form above. Create `LoginServlet.java` that reads username and password. If username is 'admin' and password is 'tops123', print 'Welcome, Admin!' in an H2 tag. Otherwise print 'Invalid credentials. Try again.' Deploy on Tomcat and test both paths.

HttpServletRequest Method	Returns	Use Case
<code>getParameter(String name)</code>	String (or null)	Single form field value

HttpServletRequest Method	Returns	Use Case
getParameterValues(String name)	String[]	Checkbox / multi-select field
getParameterNames()	Enumeration<String>	List all field names in request
getHeader(String name)	String	Read a specific HTTP header
getMethod()	String (GET / POST)	Check which HTTP method was used
getRemoteAddr()	String (IP)	Log the client IP address

CASE STUDY — Zerodha Login Security

Context — Zerodha is India's largest stockbroker with over 1.3 crore active users, headquartered in Bengaluru.

Challenge — Login forms that use GET requests expose passwords in the URL — visible in browser history, server logs, and network traffic. Zerodha needed secure credential transmission.

What they did — Zerodha's login form uses HTTP POST with HTTPS, ensuring credentials travel in the encrypted request body and never in the URL. The backend Servlet validates credentials against a hashed password in the database.

Result — No credential leaks from URL exposure. Combined with 2FA, Zerodha's login is considered one of the most secure in Indian fintech.

Takeaway — Using POST for login forms — exactly what you built today — is a non-negotiable security practice, not just a convention.

Source: Zerodha Tech Blog, 2023

WHERE TO GO NEXT

You can now read anything a user submits — form fields, headers, and request metadata.

Explore deeper — Java EE HttpServletRequest API docs:

<https://docs.oracle.com/javaee/7/api/javax/servlet/http/HttpServletRequest.html>

Try on your own — Extend the login Servlet to also print the user's browser name and IP address on the welcome page using request headers.

Connect the dots — Next session covers ServletConfig and ServletContext — how to store configuration values so your Servlet never needs hardcoded settings.

Going further — Research XSS (Cross-Site Scripting) attacks — what happens when you print `getParameter()` output directly into HTML without sanitising it.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java Servlet expert. Explain the difference between HTTP GET and POST in simple Hinglish. Use

the example of searching for a product on Meesho (GET) versus placing an order (POST). Keep it under 150 words.

SESSION

04

60 MIN · SERVLETS

ServletConfig & ServletContext

◆ Learning Objectives

- Explain the difference between ServletConfig and ServletContext
- Read Servlet-specific init parameters from web.xml using ServletConfig
- Read application-wide parameters from web.xml using ServletContext
- Store and retrieve shared objects across Servlets using ServletContext attributes
- Apply best practices for externalising configuration such as DB connection strings

01 Why Externalise Configuration?

Hardcoding values like database URLs, usernames, or API keys inside Java code is a serious mistake. When you move from development to production, you must change the code, recompile, and redeploy. Externalising config into web.xml means you change one file — no recompile needed.

CONCEPT — Config vs Hardcoding

Imagine every TOPS branch has a different database server IP. If you hardcode the IP in Java, you need a separate build for each branch. With ServletConfig or environment variables, you change the config file and the same .war file works everywhere.

02 ServletConfig — Per-Servlet Parameters

ServletConfig holds initialisation parameters specific to one Servlet. It is created by Tomcat when the Servlet is initialised and passed to `init()`. These params are only visible to that one Servlet — not shared with others in the application.

```
<!-- web.xml - inside <servlet> block -->
<init-param>
  <param-name>reportFormat</param-name>
  <param-value>PDF</param-value>
</init-param>

// Inside the Servlet
String format = getInitParameter("reportFormat");
Returns 'PDF' — shorthand method
```

Servlet-specific parameter
Parameter key
Parameter value

03 ServletContext — Application-Wide Parameters

ServletContext is shared across the entire web application — all Servlets, JSPs, and filters can read it. It is created once when Tomcat starts and destroyed when Tomcat stops. Use it for application-wide config like database connection strings or site-wide settings.

```
<!-- web.xml – OUTSIDE <servlet> block -->
<context-param>                                Application-wide parameter
  <param-name>dbUrl</param-name>
  <param-value>jdbc:mysql://localhost:3306/topsdb</param-value>
</context-param>

// Read from any Servlet
ServletContext ctx = getServletContext();        Get shared application context
String dbUrl = ctx.getInitParameter("dbUrl");    Read context-level param
ctx.setAttribute("appName", "TOPS Portal");      Store a shared Java object
String name = (String) ctx.getAttribute("appName");
// Retrieve it from any Servlet
```

ServletConfig

Specific to one Servlet. Set as `<init-param>` inside `<servlet>`. Accessed via `getInitParameter()`. Lifetime: as long as that Servlet is alive.

ServletContext

Shared by the entire application. Set as `<context-param>` outside `<servlet>`. Accessed via `getServletContext()`. Lifetime: entire web app runtime.

PRO TIP — Use Context for DB Connection Pools

A common pattern is to create a database connection pool (like HikariCP) in a `ServletContextListener` at startup, store it as a ServletContext attribute, and then retrieve it in any Servlet that needs a DB connection. This avoids creating a new connection per request — a very expensive operation.

04 ServletContext as a Shared Noticeboard

Beyond init parameters, `ServletContext.setAttribute()` lets you store any Java object and share it across all Servlets. This is useful for global counters, cached data, or shared configuration objects that multiple Servlets need to access simultaneously.

TRY THIS LIVE

In `web.xml`, add a `context-param` with name 'instituteName' and value 'TOPS Technologies'. Create two different Servlets that both read this value using `getServletContext().getInitParameter()`. Observe that both return the same value without any hardcoding in either class.

Feature	ServletConfig	ServletContext
Scope	Single Servlet	Entire application
Defined in web.xml	<init-param> inside <servlet>	<context-param> outside <servlet>
Read via	getInitParameter()	getServletContext().getInitParameter()
Store custom object	Not supported	setAttribute() / getAttribute()
Lifetime	Servlet lifecycle	Application lifecycle

CASE STUDY — MakeMyTrip Configuration Management

Context — MakeMyTrip is India's leading online travel company, processing thousands of API calls to airline partners, hotel chains, and payment gateways simultaneously.

Challenge — Different API endpoints, credentials, and timeout values for each partner cannot be hardcoded. Deploying a fix for one partner should not require recompiling the whole application.

What they did — Configuration values are stored externally in context params or environment variables. A shared application-context object holds all partner config, accessible by any Servlet or service class.

Result — Partner credentials can be rotated and API endpoints updated without a redeployment. Mean-time-to-fix configuration issues dropped from hours to minutes.

Takeaway — Externalising config via ServletContext — what you practised today — is not just a best practice; it is a production requirement.

Source: MakeMyTrip Engineering Blog, 2022

WHERE TO GO NEXT

You can now drive Servlet behaviour through configuration files instead of hardcoded values.

Explore deeper — The Twelve-Factor App methodology on config: <https://12factor.net/config>

Try on your own — Add a context-param for 'maxLoginAttempts' set to 3. In your LoginServlet, read this value and display a 'Too many attempts' message after 3 failures using a session counter.

Connect the dots — Next session covers sendRedirect() and RequestDispatcher — controlling where the user goes after the Servlet processes their request.

Going further — Research JNDI (Java Naming and Directory Interface) — the enterprise-grade way to store and retrieve config, including DataSources, in a Tomcat server.

SESSION

05

60 MIN · SERVLETS

Response Handling & Redirection

◆ Learning Objectives

- Set response content type, status codes, and headers using HttpServletResponse
- Use sendRedirect() to redirect the user to a different URL
- Use RequestDispatcher.forward() to forward a request within the server
- Explain the key differences between redirect and forward
- Build a login flow that uses forward on success and redirect on failure

01 Controlling the HTTP Response

The HttpServletResponse object is your tool to build and send the HTTP response back to the browser. You can set the content type (HTML, JSON, PDF), the status code (200, 404, 500), custom headers, and the response body itself.

```
response.setContentType("text/html; charset=UTF-8");
```

Tell browser the response is HTML

```
response.setContentType("application/json");
```

 For JSON API responses

```
response.setStatus(HttpServletResponse.SC_OK);
```

200 — request succeeded

```
response.setStatus(HttpServletResponse.SC_NOT_FOUND);
```

404 — resource not found

```
response.setHeader("Cache-Control", "no-cache");
```

Prevent browser from caching this page

02 sendRedirect() — External Redirect

response.sendRedirect(url) tells the browser: 'Go to this new URL.' The browser receives a 302 response, then makes a brand-new HTTP GET request to the new URL. The URL in the address bar changes. Use this after a form submission to prevent duplicate submissions on page refresh.

```
// After successful login, redirect to dashboard
```

```
response.sendRedirect("dashboard.jsp");
```

 Redirect to relative URL

```
response.sendRedirect("https://www.tops-int.com");
```

Redirect to absolute external URL

```
// Browser makes a NEW GET request — original POST data is LOST
```

CONCEPT — Post/Redirect/Get (PRG) Pattern

When a user submits an order form and you respond directly with the confirmation page, refreshing the browser will resubmit the order — creating a duplicate. The PRG pattern fixes this: process the POST, then `sendRedirect()` to a GET page. On refresh, only the safe GET is repeated. Every major Indian e-commerce site (Flipkart, Amazon IN) uses this pattern.

03 `RequestDispatcher.forward()` — Server-Side Forward

`forward()` transfers the request and response objects to another Servlet or JSP within the same application, entirely on the server side. The browser URL does not change. The original request data (parameters, attributes) is preserved and available at the destination.

```
// Forward to a JSP and pass data via request attribute
request.setAttribute("user", userObject);           Attach data to the request
RequestDispatcher rd = request.getRequestDispatcher("welcome.jsp");
    Get dispatcher for target JSP
rd.forward(request, response);                       Forward — URL stays the same, data preserved

// To include another resource's output inline:
rd.include(request, response);                       Embeds the included resource's output
```

`sendRedirect()`

Browser makes a new request. URL in address bar changes. Original request attributes are LOST. Can redirect to external URLs. Two round trips (302 + new GET). Use for: post-login redirect, external links, PRG pattern.

`RequestDispatcher.forward()`

Server internally forwards the request. URL stays the same. Request attributes are PRESERVED. Only works within the same web app. Single round trip. Use for: passing data from Servlet to JSP (MVC).

PRO TIP — `forward()` Is the Heart of MVC

In the MVC pattern you will build in Session 9, the Servlet (Controller) always uses `forward()` to pass data to the JSP (View). This keeps the URL clean and passes the Model data via `request.setAttribute()` — the foundation of every Java MVC framework.

TRY THIS LIVE

Upgrade your `LoginServlet` from Session 3. On successful login: use `request.setAttribute('username', username)` then `RequestDispatcher.forward()` to `welcome.jsp`. On failed login: use `response.sendRedirect('login.html?error=1')`. Observe that the URL changes on failure but stays on `LoginServlet` URL on success.

Scenario	Use	Reason
After form submit, go to confirmation page	<code>sendRedirect()</code>	PRG pattern — prevents duplicate submissions
Pass data from Servlet to JSP for display	<code>forward()</code>	Preserves request attributes
Redirect to Razorpay payment gateway	<code>sendRedirect()</code>	External URL, new request needed
Show error page for unhandled exception	<code>forward()</code>	Internal, no URL change needed
After logout, go to login page	<code>sendRedirect()</code>	Clear request context, start fresh GET

CASE STUDY — Paytm Checkout Flow

Context — Paytm processes millions of payment transactions daily across India, from mobile recharges to utility bills and online shopping.

Challenge — After a payment POST is processed, if the user refreshes the confirmation page, the browser tries to resubmit the POST — potentially charging the user twice.

What they did — Paytm's checkout flow uses the Post/Redirect/Get pattern. After the payment POST is processed, the server sends a `sendRedirect()` to a unique order confirmation URL. Refreshing only reloads the GET confirmation page.

Result — Duplicate payment submissions from browser refresh are eliminated. Each transaction is idempotent. 'Double charge' complaints dropped significantly.

Takeaway — `sendRedirect()` after a POST is not a nice-to-have — in payment flows, it is a safety mechanism.

Source: Paytm Engineering, 2022

WHERE TO GO NEXT

You now control exactly where users go after every action — forward for data passing, redirect for navigation.

Explore deeper — MDN HTTP Redirections guide:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Redirections>

Try on your own — Build a multi-step registration flow: Page 1 collects name -> Servlet stores in session -> `forward()` to Page 2 for email -> final Servlet shows a summary of both fields.

Connect the dots — Next session is Session Tracking — how to remember who the user is across multiple requests using cookies and `HttpSession`.

Going further — Research HTTP 301 vs 302 vs 307 redirect codes and how search engines (Google) treat them differently for SEO.

SESSION

06

60 MIN · SERVLETS

Session Tracking

◆ Learning Objectives

- Explain why HTTP is stateless and why session tracking is necessary
- Implement session tracking using Cookies
- Implement session tracking using the HttpSession API
- Describe URL rewriting and hidden form fields as fallback techniques
- Build a login system that keeps the user authenticated across multiple pages

01 The Stateless Problem

HTTP is stateless — each request is completely independent. The server has no memory of previous requests. When you add items to a cart on Amazon and move to the next page, how does the server remember your cart? It uses session tracking. Without it, you would have to log in on every single page click.

CONCEPT — Why Sessions Exist

Think of a large sabzi mandi in Ahmedabad where each vendor gives you a token when you buy something. At the exit, you show all your tokens to pay. Session tracking gives each user a unique token (session ID) to carry across requests so the server can recognise them.

02 Cookies

A Cookie is a small key-value string stored in the browser and sent automatically with every request to the same server. The server creates a cookie and sends it in the response header. The browser stores it and attaches it to every future request to that domain.

```
// Server creates and sends a cookie
Cookie userCookie = new Cookie("username", "Niral");
    Create cookie with name and value
userCookie.setMaxAge(60 * 60 * 24);           Expire after 24 hours (in seconds)
userCookie.setHttpOnly(true);                 Prevent JavaScript from reading it
response.addCookie(userCookie);               Send cookie to browser

// Server reads cookies on the next request
Cookie[] cookies = request.getCookies();       Get all cookies from browser
for (Cookie c : cookies) {
    if ("username".equals(c.getName())) {
```

```
String user = c.getValue();    Read the stored value
}
}
```

WARNING — Never Store Sensitive Data in Cookies

Cookies are stored on the user's machine. Never store passwords, credit card numbers, or session tokens in plaintext cookies. Always use `setHttpOnly(true)` and `setSecure(true)` (HTTPS only) for any authentication-related cookie.

03 HttpSession — The Preferred Method

`HttpSession` is Java's built-in server-side session. Tomcat creates a session object on the server and sends only a session ID cookie (`JSESSIONID`) to the browser. Your actual data stays securely on the server. This is the standard way to implement login sessions in Java.

```
// After successful login – create session
HttpSession session = request.getSession();    Create or retrieve existing session
session.setAttribute("loggedInUser", username);
    Store user data server-side
session.setMaxInactiveInterval(30 * 60);        Auto-expire after 30 min of inactivity

// On protected pages – check session
HttpSession session = request.getSession(false);
    false = don't create new if none exists
if (session == null || session.getAttribute("loggedInUser") == null) {
    response.sendRedirect("login.html");        Not logged in — redirect to login
    return;
}

// On logout
session.invalidate();                            Destroy session — user is logged out
```

04 Other Tracking Techniques

Two older techniques exist for environments where cookies are disabled: URL Rewriting appends the session ID to every URL as a query parameter (`?jsessionid=XYZ`). Hidden Form Fields embed the session ID as a hidden input inside every form. Both are less elegant and rarely used today but important to know.

Technique	Where Data Lives	Works Without Cookies?	Security
-----------	------------------	------------------------	----------

Technique	Where Data Lives	Works Without Cookies?	Security
Cookies	Browser storage	No	Good if HttpOnly + Secure set
HttpSession (JSESSIONID)	Server memory	Partially (URL rewrite fallback)	Best — data stays on server
URL Rewriting	URL query string	Yes	Poor — ID visible in URL and logs
Hidden Form Fields	HTML form	Yes	Poor — only works across form submits

TRY THIS LIVE

Upgrade your login system: After login, store the username in HttpSession. Create a DashboardServlet that checks for the session attribute. If found, show 'Welcome {username}'. If not, redirect to login.html. Create a LogoutServlet that calls session.invalidate() and redirects to login. Test the full login -> dashboard -> logout flow.

CASE STUDY — Flipkart Login Persistence

Context — Flipkart serves over 40 crore registered users across India, with users accessing the platform from mobile, web, and app simultaneously.

Challenge — Users expect to stay logged in across browser sessions and page navigations without re-entering credentials. Sessions must also expire automatically for security.

What they did — Flipkart uses server-side sessions for sensitive operations and persistent encrypted cookies for 'Remember Me' functionality. Sessions have strict inactivity timeouts and are invalidated on logout across all devices.

Result — Users enjoy seamless browsing while security is maintained. Session tokens are rotated after login to prevent session fixation attacks.

Takeaway — The HttpSession pattern you implemented today is the same core mechanism behind every major Indian e-commerce login.

Source: Flipkart Engineering Blog, 2023

“A session is just a way of making a stateless protocol pretend it has memory. Your job is to use that memory wisely.”

WHERE TO GO NEXT

Users can now log in, stay logged in, and log out — the foundation of any authenticated web app.

Explore deeper — OWASP Session Management Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

Try on your own — Add a protected `DashboardServlet`. If no session exists, redirect to `login.html`. After login, also display the login time using `session.getCreationTime()`.

Connect the dots — Next session introduces JSP — the View layer where you will present session data beautifully instead of hardcoding HTML in your Servlet.

Going further — Research JWT (JSON Web Tokens) — the modern stateless alternative to server-side sessions used in REST APIs and mobile backends.

SESSION

07

60 MIN · JSP

JSP Basics

◆ Learning Objectives

- Explain what JSP is and how it differs from writing HTML in a Servlet
- Describe the JSP lifecycle and how Tomcat converts a JSP file into a Servlet
- Use JSP directives (page, include, taglib) correctly
- Write scriptlets, expressions, and declarations in JSP
- Use Expression Language (EL) to display data without scriptlets

01 What Is JSP and Why Use It?

JSP (JavaServer Pages) lets you write HTML with embedded Java code. Instead of writing ugly `out.println()` statements in a Servlet to produce HTML, you write HTML directly and embed dynamic parts with special JSP tags. Tomcat automatically converts your JSP file into a Servlet behind the scenes.

Generating HTML in Servlet

```
out.println("<html>");
out.println("<h1>" + name + "</h1>");
out.println("</html>");
```

Very hard to read, maintain, or style. UI team cannot work on it.

Generating HTML in JSP

```
<html><body>
<h1>Welcome, ${name}!</h1>
</body></html>
```

Clean HTML with minimal Java. Easy for designers to understand, style, and maintain.

CONCEPT — JSP Is Still a Servlet

When a browser first requests a .jsp file, Tomcat translates it into a Java Servlet class, compiles it, and runs it. On subsequent requests, the compiled Servlet is reused (very fast). If you change the JSP, Tomcat re-translates. This translation step only happens once — JSP is not interpreted on every request.

02 JSP Lifecycle

Phase	What Happens	Analogy
Translation	Tomcat converts .jsp to .java Servlet	Chef reads the recipe for the first time
Compilation	Tomcat compiles .java to .class	Chef preps the kitchen tools
Initialization	<code>jspInit()</code> called once	Kitchen opens for the day

Phase	What Happens	Analogy
Request Processing	_jspService() called per request	Chef cooks each customer's order
Destruction	jspDestroy() called on shutdown	Kitchen closes for the night

03 JSP Directives

Directives give instructions to the JSP engine at translation time. They do not produce output — they configure the JSP. There are three types: page (sets page-level settings), include (inserts another file at translation time), and taglib (imports a tag library like JSTL).

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
```

Page directive — sets content type and encoding

```
<%@ page import="java.util.List, java.util.ArrayList" %>
```

Import Java classes for use in the page

```
<%@ page errorPage="error.jsp" %>
```

Redirect any JSP exception to error.jsp

```
<%@ include file="header.jsp" %>
```

Statically include header.jsp at translation time

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

Import JSTL core tag library

04 Scriptlets, Expressions & Declarations

JSP provides three ways to embed Java in HTML. Scriptlets run Java code. Expressions output a value directly into the HTML. Declarations define methods or class-level variables. In modern JSP, EL is preferred over scriptlets — scriptlets mix logic with presentation and make code hard to maintain.

```
<% String city = "Ahmedabad"; %>
```

SCRIPTLET — runs Java code (no direct output)

```
<%= city.toUpperCase() %>
```

EXPRESSION — outputs value into HTML: AHMEDABAD

```
<%! int counter = 0; %>
```

DECLARATION — class-level variable (shared across requests)

```
${sessionScope.loggedInUser}
```

EL — reads 'loggedInUser' from session scope

```
${param.username}
```

EL — reads 'username' request parameter

```
${requestScope.student.name}
```

EL — reads name property of student object in request scope

CONCEPT — EL Scopes

Expression Language auto-searches four scopes in order: page -> request -> session -> application. \${name} finds the first 'name' attribute in any scope. Be explicit with \${sessionScope.name} or \${requestScope.name} to avoid accidental scope conflicts.

TRY THIS LIVE

Create welcome.jsp. In your LoginServlet, after successful login, do `request.setAttribute('username', username)` then forward to welcome.jsp. In welcome.jsp, display: `<h1>Welcome, ${requestScope.username}</h1>` and `<p>Time: <%= new java.util.Date() %></p>`. Compare the clean EL syntax vs the scriptlet approach and notice which is easier to read.

05 JSP Implicit Objects

JSP pre-creates several objects you can use directly without declaring them. These are called implicit objects. The most commonly used are request, response, session, application (ServletContext), out (JspWriter), and pageContext.

Implicit Object	Type	Maps To
request	HttpServletRequest	The current HTTP request
response	HttpServletResponse	The HTTP response object
session	HttpSession	The current user's session
application	ServletContext	The whole web app shared context
out	JspWriter	Output stream to write HTML
pageContext	PageContext	Access to all scopes and implicit objects

CASE STUDY — Naukri.com Job Listings (InfoEdge India)

Context — Naukri.com, India's largest job portal with over 7 crore registered jobseekers, was originally built on JSP-based Java technology.

Challenge — Displaying thousands of dynamic job listings — each personalised to the logged-in user's profile, location, experience, and skills — required a clean, maintainable view layer.

What they did — Naukri used JSP as the view layer with Servlets as controllers passing job data via `request.setAttribute()`. JSP pages used EL to render data cleanly. The separation allowed backend and UI teams to work independently.

Result — The JSP+Servlet MVC architecture scaled to millions of daily searches with the UI team and backend team working in parallel without conflicts.

Takeaway — JSP is not just for learning — it was the backbone of production systems serving crores of Indians.

Source: InfoEdge India Engineering, 2020

WHERE TO GO NEXT

You can now create dynamic HTML pages that display Java data cleanly — no more `out.println()` nightmares.

Explore deeper — Oracle JSP Tutorial: <https://docs.oracle.com/javaee/7/tutorial/servlets010.htm>

Try on your own — Create a `profile.jsp` that displays the logged-in user's username from session, the current date/time, and the user's IP address. Refactor all scriptlets to use EL only — observe how much cleaner it becomes.

Connect the dots — Next session covers JSP Action Tags — built-in tags that let you include pages, use JavaBeans, and set properties without writing any Java scriptlets.

Going further — Research Thymeleaf and Freemarker — modern Java template engines that replace JSP in Spring Boot projects and are 100% HTML5 compatible.

SESSION

08

60 MIN · JSP

JSP Actions & Tags

◆ Learning Objectives

- Use standard JSP action tags: `jsp:include`, `jsp:forward`, `jsp:useBean`, `jsp:setProperty`, `jsp:getProperty`
- Differentiate between static include (directive) and dynamic include (action tag)
- Create a reusable page layout using `jsp:include` for shared header and footer
- Introduce JSTL core tags (`c:if`, `c:forEach`, `c:set`) as the modern alternative to scriptlets
- Build a dynamic student list table using JSTL `forEach`

01 Standard JSP Action Tags

JSP Action Tags are built-in XML-style tags that control the JSP engine's behaviour at request time. Unlike directives (which work at translation time), action tags run when the page is requested. They are always available in every JSP — no import needed.

Action Tag	Purpose	Example Use
<code><jsp:include></code>	Dynamically include another JSP/HTML at runtime	Add header.jsp and footer.jsp to every page
<code><jsp:forward></code>	Forward request to another JSP or Servlet	Forward to login.jsp if not authenticated
<code><jsp:useBean></code>	Create or retrieve a JavaBean in a scope	Use a Student bean from request scope
<code><jsp:setProperty></code>	Set a property of a JavaBean	Set student.name from form parameter
<code><jsp:getProperty></code>	Print a JavaBean property value	Display student.email in the page

02 Dynamic Include vs Static Include

There are two ways to include another file in JSP. The directive `<%@ include file='x.jsp' %>` is static — it copies the content at translation time (one compiled Servlet). The `<jsp:include page='x.jsp'>` action is dynamic — it invokes the included file at runtime, allowing the included page to change independently.

Static Include (Directive)

```
<%@ include file='header.jsp' %>
```

Inlined at translation time. Changes to header.jsp require re-translation of all pages that include it.

Faster at runtime. Use for content that rarely changes.

Dynamic Include (Action Tag)

```
<jsp:include page='header.jsp' />
```

Included at request time. Changes to header.jsp are reflected immediately. Slightly slower. Use for content that changes frequently or needs parameters.

```
<!-- Include a shared header and pass a parameter -->
<jsp:include page='common/header.jsp' >           Include header dynamically
<jsp:param name='pageTitle' value='Dashboard' />
    Pass a parameter to the included page
</jsp:include>

<!-- Main content here -->

<jsp:include page='common/footer.jsp' />           Include footer
```

TRY THIS LIVE

Create header.jsp with a nav bar showing 'TOPS Portal | \${param.pageTitle}'. Create footer.jsp with '(c) 2026 TOPS Technologies'. Create dashboard.jsp that uses <jsp:include> to add both. Change the footer text and refresh dashboard.jsp — changes appear instantly because the include is dynamic.

03 JavaBeans & jsp:useBean

A JavaBean is a Java class with a no-arg constructor, private fields, and public getters/setters. JSP can create and access JavaBean objects using <jsp:useBean>, <jsp:setProperty>, and <jsp:getProperty> — without writing any scriptlet Java code.

```
<!-- Retrieve 'student' bean from request scope -->
<jsp:useBean id='student' class='com.tops.Student' scope='request' />
    Create or retrieve bean

<!-- Auto-populate all matching form fields into bean properties -->
<jsp:setProperty name='student' property='*' />
    Maps form params to matching bean properties

Name: <jsp:getProperty name='student' property='name' />
    Outputs student.getName()
Email: <jsp:getProperty name='student' property='email' />
    Outputs student.getEmail()
```

04 JSTL Core Tags — The Modern Alternative

JSTL (JavaServer Pages Standard Tag Library) provides tags for common tasks: conditionals, loops, variable setting, and URL handling. JSTL replaces scriptlets with clean, readable tags. It is the industry standard for JSP view development — no Java code inside the HTML.

```
<%@ taglib prefix='c' uri='http://java.sun.com/jsp/jstl/core' %>
```

Import JSTL core at top of JSP

```
<c:if test='${not empty sessionScope.loggedInUser}'>
```

Show only if user is logged in

```
<p>Welcome, ${sessionScope.loggedInUser}!</p>
```

```
</c:if>
```

```
<c:forEach var='student' items='${requestScope.studentList}'>
```

Loop over list forwarded by Servlet

```
<tr>
```

```
<td>${student.name}</td><td>${student.email}</td>
```

```
</tr>
```

```
</c:forEach>
```

CONCEPT — Why Avoid Scriptlets in JSP?

Scriptlets (`<%...%>`) mix Java and HTML in one file — making it impossible for front-end designers to work on the page, hard to unit test, and a maintenance nightmare. JSTL tags and EL look like HTML, so designers can work on them directly. Rule of thumb: if you are writing Java code inside JSP, move it to the Servlet instead.

CASE STUDY — HDFC Bank NetBanking Portal

Context — HDFC Bank's NetBanking portal serves crores of customers with account statements, fund transfers, and bill payments — one of India's busiest financial web applications.

Challenge — The portal needed to display dynamic account data (transaction lists, balances, EMI schedules) in complex HTML tables while keeping UI code maintainable by a separate front-end team.

What they did — HDFC NetBanking JSP pages use JSTL `forEach` to render transaction rows, `c:if` for conditional alerts (low balance, upcoming EMI), and `jsp:include` for consistent header/footer layout across all pages.

Result — Front-end developers modify HTML/CSS without touching Java code. Backend developers focus only on the Servlet/DAO layer. Page load performance was maintained with caching at the Servlet layer.

Takeaway — JSTL and `jsp:include` are not just conveniences — they are the architecture patterns that made large Indian banking portals maintainable.

Source: HDFC Technology Division, 2021

“JSP's job is to display data beautifully — not to compute it. Keep logic in the Servlet, presentation in JSP.”

WHERE TO GO NEXT

Your JSP pages are now clean, reusable, and loop-capable — ready for real data from a Servlet.

Explore deeper — JSTL 1.2 Tag Reference: <https://docs.oracle.com/javaee/7/tutorial/jsf-el001.htm>

Try on your own — Create a Servlet that builds an ArrayList of 5 Student objects and forwards it to students.jsp. In students.jsp, use c:forEach to display all students in an HTML table with alternating row colours using c:if and varStatus.

Connect the dots — Next session is the big one — the full MVC pattern: Servlet as Controller, JSP as View, and Java classes as the Model, all working together.

Going further — Research Spring's Thymeleaf templates — the modern replacement for JSP in Spring Boot that is fully HTML5 compatible and designer-friendly.

SESSION

09

60 MIN · ARCHITECTURE

MVC Pattern Using Servlet & JSP

◆ Learning Objectives

- Explain the Model-View-Controller (MVC) design pattern and its core benefits
- Map Servlet to Controller, JSP to View, and JavaBeans/DAOs to Model in a Java web app
- Implement the complete MVC flow for a student registration feature end-to-end
- Use `request.setAttribute()` and `RequestDispatcher.forward()` to pass Model data to the View
- Organise a project folder structure following MVC conventions

01 What Is MVC?

MVC (Model-View-Controller) is the most important design pattern in web development. It separates your application into three layers: the Model (data and business logic), the View (what the user sees), and the Controller (the layer in between that handles requests and coordinates the other two). This separation makes applications easier to build, test, and maintain.

CONCEPT — MVC in Java Web

Controller = Servlet (handles HTTP requests, calls Model, chooses View). Model = Java POJO + DAO class (fetches data from DB, applies business logic). View = JSP (renders data as HTML for the user). Each layer has one job and does not do the others'.

Without MVC

Database code, business logic, and HTML are all mixed in one Servlet or JSP. Changing the UI requires editing the same file as the DB logic. Adding a new developer causes conflicts. Testing is nearly impossible.

With MVC

Servlet handles routing only. Java classes handle logic and DB. JSP handles HTML only. A designer can change the View without touching Java. A backend developer can change logic without touching HTML.

02 The MVC Flow Step by Step

Step	Who Acts	What Happens
1. Request	Browser	User submits student registration form (POST /RegisterStudentServlet)
2. Controller	Servlet	Reads form params, creates Student POJO, calls DAO to save to DB

Step	Who Acts	What Happens
3. Model/DAO	StudentDAO.java	Inserts student record into MySQL, returns success/failure
4. Pass to View	Servlet	Sets result as request attribute, calls <code>RequestDispatcher.forward('success.jsp')</code>
5. View	success.jsp	Reads attributes via EL, renders 'Student registered successfully' page

03 Building the Model — Student POJO & DAO

```
// Student.java – POJO (Model)
public class Student {
    private String name, email, course;
    // Constructors, getters, setters...
}

// StudentDAO.java – Data Access Object
public class StudentDAO {
    public boolean registerStudent(Student s) {
        // JDBC INSERT into students table (Session 11)
        return true; // Placeholder — return true for now
    }
}
```

04 Building the Controller — RegisterStudentServlet

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    // 1. Read form data
    String name = req.getParameter("name");
    String email = req.getParameter("email");
    // 2. Create Model object
    Student student = new Student(name, email, "Java Full Stack");
    // 3. Call DAO
    boolean success = new StudentDAO().registerStudent(student);
    // 4. Set attributes for JSP
    req.setAttribute("student", student);
}
```

```
req.setAttribute("success", success);
// 5. Forward to View
req.getRequestDispatcher("success.jsp").forward(req, res);
```

05 Building the View — success.jsp

```
<%@ taglib prefix='c' uri='http://java.sun.com/jsp/jstl/core' %>
<c:if test='${requestScope.success}'>
    <h2>Registration Successful!</h2>
    <p>Welcome, ${requestScope.student.name}!</p>
</c:if>
<c:if test='${not requestScope.success}'>
    <h2>Registration Failed. Please try again.</h2>
</c:if>
```

PRO TIP — Recommended Project Folder Structure

```
src/com/tops/model/ -> Student.java, Course.java
src/com/tops/dao/ -> StudentDAO.java
src/com/tops/controller/ -> RegisterStudentServlet.java
WebContent/views/ -> success.jsp, login.jsp, dashboard.jsp
WebContent/static/ -> css/, js/, images/
```

TRY THIS LIVE

Build the complete student registration MVC flow: register.html form -> RegisterStudentServlet (reads params, creates Student object) -> forward to success.jsp (displays student name using EL). Skip the real DB for now — just store the student in a request attribute and display it in JSP using `${requestScope.student.name}`.

CASE STUDY — Byju's Learner Registration Flow

Context — Byju's (Think & Learn Pvt. Ltd.), India's largest ed-tech company, handled millions of student registrations from across India on its Java backend.

Challenge — The registration flow involved multiple steps — form validation, duplicate email check, account creation, email confirmation — and mixing this logic with HTML presentation made the system unmaintainable.

What they did — Byju's backend uses an MVC architecture where Controllers handle routing, Service/DAO classes handle DB operations, and View templates handle only presentation. This is implemented via Spring MVC.

Result — Backend and frontend teams work on separate layers simultaneously. New features such as OTP verification are added to the Controller layer without changing the View. The system handles 100,000+ registrations per day.

Takeaway — MVC is not just a pattern — it is the organisational structure that lets teams scale without stepping on each other.

Source: Byju's Engineering Blog, 2022

WHERE TO GO NEXT

You have built the three-layer MVC architecture that powers every major Java web application.

Explore deeper — Martin Fowler on MVC: <https://martinfowler.com/eaDev/uiArchs.html>

Try on your own — Extend the MVC to a course listing page: Create a CourseDAO with a hardcoded list of 5 courses, forward it from a Servlet to courses.jsp, and display in a JSTL forEach table.

Connect the dots — Next session adds form validation and error pages — making your MVC flow bulletproof against bad user input.

Going further — Research the Front Controller pattern — how Spring MVC uses a single DispatcherServlet rather than one Servlet per feature, and how @Controller annotations map URLs automatically.

SESSION

10

60 MIN · SERVLETS

Form Validation & Error Pages

◆ Learning Objectives

- Implement server-side form validation for null, empty, length, and format checks
- Return meaningful validation error messages back to the form without losing user input
- Configure a custom JSP error page using the `errorPage` directive
- Handle Java exceptions in Servlets gracefully and display user-friendly messages
- Distinguish between client-side and server-side validation and explain why both are needed

01 Why Server-Side Validation Is Non-Negotiable

Client-side validation (HTML5 required, JavaScript checks) is easily bypassed — a developer can disable JavaScript in the browser or send a raw HTTP request using Postman. Server-side validation in your Servlet is the real security gate. Never trust data from the browser.

Client-Side Validation (JS/HTML5)

Runs in the browser before the request is sent. Gives instant feedback — good UX. Can be bypassed by disabling JS or using tools like Postman. Always use this for UX, never as the only check.

Server-Side Validation (Servlet)

Runs in Java on the server. Cannot be bypassed — the server always validates. Is the authoritative check. Required for all production applications.

02 Common Validation Checks in Servlets

```
Map<String, String> errors = new HashMap<>();
```

 Store errors by field name

```
String name = request.getParameter("name");
String email = request.getParameter("email");
String phone = request.getParameter("phone");
```

```
// Null / empty check
if (name == null || name.trim().isEmpty())
    errors.put("name", "Name is required");
```

```
// Length check
else if (name.length() < 2 || name.length() > 50)
    errors.put("name", "Name must be 2-50 characters");
```

```
// Email format (regex)
if (!email.matches("^([\\w.-]+@[\\w.-]+\\.([a-zA-Z]{2,}$)"))
errors.put("email", "Enter a valid email address");

// Indian mobile number – 10 digits, starts 6-9
if (!phone.matches("[6-9][0-9]{9}"))
errors.put("phone", "Enter a valid 10-digit Indian mobile number");
```

03 Returning Errors to the Form

When validation fails, do not redirect — redirect loses all data. Instead, use `forward()` back to the form JSP. Store the errors and the original input in request attributes so the user sees their entries pre-filled and clear error messages next to each field.

```
if (!errors.isEmpty()) {
    request.setAttribute("errors", errors);
    request.setAttribute("formData", request.getParameterMap());
    request.getRequestDispatcher("register.jsp").forward(request, response);
    return;
}
```

Validation failed
Pass error map to JSP
Pass back user's input to pre-fill
Stop further processing

```
<!-- In register.jsp – show error next to field -->
<input type='text' name='name'
    value='${formData["name"][0]}' />
<c:if test='${not empty errors["name"]}'>
<span class='error'>${errors['name']}</span>
</c:if>
```

Pre-fill the user's input
Show error message in red

TRY THIS LIVE

Add validation to your student registration Servlet. Check: name (not empty, 2–50 chars), email (valid format), phone (10-digit Indian number). If validation fails, forward back to `register.jsp` with errors and pre-filled values. Test with: empty form, invalid email, phone with letters, and correct data.

04 JSP Error Pages

When an unhandled exception occurs in a JSP, Tomcat shows an ugly stack trace to the user — a security risk and a terrible experience. Configure a custom error JSP to show a friendly message instead.

```
<!-- In any JSP that might throw an error: -->
<%@ page errorPage='error.jsp' %>                                Redirect exceptions to error.jsp

<!-- In error.jsp: -->
<%@ page isErrorPage='true' %>                                    Marks this as the error-handling page
<h2>Something went wrong. Please try again later.</h2>

<!-- Global error pages in web.xml: -->
<error-page>
  <error-code>404</error-code>
  <location>/views/404.jsp</location>
</error-page>
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/views/error.jsp</location>
</error-page>
```

WARNING — Never Show Stack Traces in Production

Exception stack traces reveal your class names, file structure, and framework versions — giving attackers a roadmap to exploit vulnerabilities. Always configure custom error pages in web.xml. Log stack traces to a file using Log4j or SLF4J — never print them to the user.

CASE STUDY — Ola Cabs Driver Registration Validation

Context — Ola Cabs (ANI Technologies) onboards both riders and driver-partners across India, with driver registration requiring phone verification, licence number, and vehicle details.

Challenge — Invalid phone numbers, fake email IDs, and incomplete vehicle details caused downstream failures in SMS OTP sending and document verification pipelines.

What they did — Ola's registration flow runs server-side validation for phone format (10-digit, valid operator prefix), PAN card format (regex), and vehicle RC number format. Errors are returned with specific field-level messages in Hindi and English.

Result — Invalid registration attempts dropped by 65%. OTP delivery success rate improved to 98.5% because only valid phone numbers reached the SMS gateway.

Takeaway — Server-side validation saves backend resources, reduces API costs, and improves data quality at the source.

Source: Ola Engineering Blog, 2022

WHERE TO GO NEXT

Your forms are now bulletproof — bad data is caught before it reaches the database or crashes your application.

Explore deeper — OWASP Input Validation Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Try on your own — Create a global error.jsp for your project. Configure web.xml to map 404 and all Java exceptions to this page. Test by visiting a non-existent URL on your Tomcat app.

Connect the dots — Session 11 adds JDBC — now that your form data is validated, you will persist it into a real MySQL database.

Going further — Research Bean Validation (JSR-380, Hibernate Validator) — the annotation-based validation framework used in Spring Boot: @NotEmpty, @Email, @Pattern, @Size.

SESSION

11

60 MIN · DATABASE

JDBC Integration with Servlets

◆ Learning Objectives

- Connect a Java Servlet to a MySQL database using JDBC
- Implement a DAO class to separate DB code from the Servlet
- Use PreparedStatement for safe parameterised SQL queries
- Fetch a list of records from MySQL and display them in a JSP table using JSTL
- Handle JDBC exceptions and close connections safely using try-with-resources

01 JDBC in the MVC Context

In Sessions 9 and 10, your Servlet registered students using hardcoded values. Now you will connect the dots: validated form data flows through the MVC Controller (Servlet), through a DAO, and into a real MySQL database using JDBC. This is the full production-style Java web flow.

CONCEPT — DAO Pattern in Web Apps

The DAO (Data Access Object) pattern puts all JDBC code in a separate class. The Servlet should never directly run SQL — it calls a DAO method like `studentDAO.getAll()` or `studentDAO.insert(student)`. This keeps the Servlet clean and makes the DB layer independently testable and replaceable.

02 JDBC Setup & Connection Utility

```
// Step 1: Add mysql-connector-j-8.x.x.jar to WEB-INF/lib/
```

Download from Maven Central

```
// DBConnection.java – reusable utility
public class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/topsdb?useSSL=false&serverTimezone=IST";
    private static final String USER = "root";
    private static final String PASS = "yourpassword";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

03 StudentDAO — INSERT and SELECT

```
// INSERT a student
public boolean insertStudent(Student s) {
    String sql = "INSERT INTO students (name, email, course) VALUES (?, ?, ?)";
    try (Connection con = DBConnection.getConnection();
        try-with-resources auto-closes connection
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, s.getName());           Bind param — prevents SQL injection
        ps.setString(2, s.getEmail());
        ps.setString(3, s.getCourse());
        return ps.executeUpdate() > 0;          Returns true if row inserted
    } catch (SQLException e) { e.printStackTrace(); return false; }
}

// SELECT all students
public List<Student> getAllStudents() {
    List<Student> list = new ArrayList<>();
    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement("SELECT * FROM students");
        ResultSet rs = ps.executeQuery()) {
        while (rs.next())
            list.add(new Student(rs.getString("name"), rs.getString("email"),
                rs.getString("course")));
    } catch (SQLException e) { e.printStackTrace(); }
    return list;
}
```

04 Wiring Servlet, DAO & JSP Together

```
// ListStudentsServlet.doGet()
List<Student> students = new StudentDAO().getAllStudents();
    Call DAO
request.setAttribute("students", students);    Pass to JSP
request.getRequestDispatcher("students.jsp").forward(request, response);
```

```
<!-- students.jsp -->
<table border='1'>
<tr><th>Name</th><th>Email</th><th>Course</th></tr>
    <c:forEach var='s' items='${students}'>
```

```
<tr><td>${s.name}</td><td>${s.email}</td><td>${s.course}</td></tr>
</c:forEach>
</table>
```

WARNING — Always Use PreparedStatement, Never String Concat

String sql = "SELECT * FROM users WHERE email = " + email + "" is vulnerable to SQL Injection. An attacker enters ' OR '1'='1 and bypasses your login entirely. Always use PreparedStatement with ? placeholders — this is non-negotiable.

TRY THIS LIVE

Create the 'topsdb' database in MySQL Workbench with a students table (id AUTO_INCREMENT, name, email, course). Wire up your Session 10 registration form to INSERT into this table via StudentDAO. Then create ListStudentsServlet that fetches all students and displays them in a Bootstrap-styled table in students.jsp.

Task	DAO Method	SQL
Register student	insertStudent(Student s)	INSERT INTO students (...)
List all students	getAllStudents()	SELECT * FROM students
Find by email	findByEmail(String email)	SELECT ... WHERE email = ?
Update course	updateCourse(int id, String course)	UPDATE students SET course = ? WHERE id = ?
Delete student	deleteById(int id)	DELETE FROM students WHERE id = ?

CASE STUDY — TOPS Technologies TopERP Student Module

Context — TOPS Technologies runs TopERP — an internal system managing 1,00,000+ students across 7 Gujarat centres, built on a Java + MySQL backend.

Challenge — Student registration, batch assignment, attendance, placement status, and certificate generation all require reading and writing to a centralised MySQL database accessible from all branches simultaneously.

What they did — TopERP uses a DAO layer where each feature (StudentDAO, BatchDAO, PlacementDAO) isolates its SQL queries. Servlets call DAOs — never writing SQL themselves. PreparedStatement prevent injection. Connection pooling handles concurrent requests from all branches.

Result — The system reliably handles 200+ concurrent users during peak admission seasons. SQL injection

is prevented by design. Adding a new report is as simple as adding a new DAO method.

Takeaway — The DAO + Servlet + JSP pattern you built today is exactly the architecture behind the ERP system you will encounter as a TOPS student.

Source: TOPS Technologies Internal Architecture, 2024

WHERE TO GO NEXT

Your web app now reads from and writes to a real database — it is a genuine full-stack Java web application.

Explore deeper — HikariCP connection pool (fastest Java connection pool):

<https://github.com/brettwooldridge/HikariCP>

Try on your own — Add Edit and Delete buttons to each row in students.jsp. Create UpdateStudentServlet and DeleteStudentServlet that update/delete the record by ID and redirect back to the list.

Connect the dots — Final session ties together everything from this module and maps each concept directly to its Spring MVC equivalent — preparing you for the next module.

Going further — Research connection pooling — why creating a new DB connection per request is expensive, and how HikariCP or c3p0 solve this with pre-created connection pools.

SESSION

12

60 MIN · REVIEW

Servlet & JSP Review

◆ Learning Objectives

- Consolidate understanding of the complete Servlet and JSP lifecycle end-to-end
- Trace the full MVC + JDBC flow from browser request to database and back to the browser
- Identify the most common mistakes and best practices across all topics in this module
- Map every Servlet/JSP concept directly to its Spring MVC equivalent
- Prepare confidently for the transition to Hibernate and Spring in the next modules

01 What You Have Built — Module Recap

Over 11 sessions you have built a complete Java web application from scratch — HTTP request handling, Servlet lifecycle, session management, JSP views, MVC architecture, form validation, and database persistence. This session connects all the dots and prepares you for the next evolution: Spring Framework.

Session	Topic	Maps to Spring Concept
01	Web Architecture & HTTP	Same — Spring uses HTTP; DispatcherServlet is the front gate
02	Servlet Lifecycle	@PostConstruct / @PreDestroy on Spring beans
03	doGet / doPost / Parameters	@GetMapping / @PostMapping / @RequestParam
04	ServletConfig / ServletContext	@Value / ApplicationContext / @Bean
05	sendRedirect / forward()	RedirectAttributes / return 'redirect:' / ModelAndView
06	HttpSession	HttpSession (same API) / Spring Security context
07	JSP + EL	Thymeleaf \${...} / JSP still supported in Spring MVC
08	JSTL Tags	Thymeleaf th:each / th:if — same concept, cleaner syntax
09	MVC Pattern	@Controller / @Service / @Repository layers in Spring

Session	Topic	Maps to Spring Concept
10	Validation	@Valid / BindingResult / Hibernate Validator annotations
11	JDBC + DAO	Spring Data JPA / @Repository / JdbcTemplate
12	Review	You are ready for Spring Boot

02 End-to-End Flow Recap

CONCEPT — The Full Java Web Request Lifecycle

1. Browser sends HTTP POST to /RegisterStudentServlet
2. Tomcat creates HttpServletRequest and HttpServletResponse
3. Servlet.service() calls doPost()
4. doPost() reads parameters: name, email, course
5. Server-side validation runs — if errors, forward back to form
6. Student POJO created from valid data
7. StudentDAO.insertStudent() runs a PreparedStatement INSERT into MySQL
8. Servlet sets 'student' as request attribute
9. RequestDispatcher.forward() sends control to success.jsp
10. JSP uses EL to render \${student.name} into HTML
11. HTTP 200 response sent to browser
12. Session attribute tracks the logged-in user across pages

03 Common Mistakes & How to Fix Them

Mistake	Symptom	Fix
String concat in SQL	SQL Injection vulnerability	Always use PreparedStatement with ?
Not null-checking getParameter()	NullPointerException in Servlet	Check != null && !isEmpty() before use
sendRedirect() when passing data to JSP	Request attributes lost — JSP shows null	Use forward() when passing data to JSP
Stack traces shown to users	Security vulnerability + terrible UX	Configure errorPage in web.xml, log to file
Sensitive data in cookies	Data exposed to browser / JS	Use HttpSession — data stays on server

Mistake	Symptom	Fix
New DB connection per request	Performance degrades under load	Use connection pooling with HikariCP
Business logic in JSP scriptlets	Unmaintainable, untestable code	Move all logic to Servlet or DAO layer

04 Servlet/JSP -> Spring MVC Bridge

Spring MVC is not a replacement for what you learned — it is an automated, convention-driven wrapper around the same concepts. `DispatcherServlet` is a single Servlet Spring uses instead of your multiple Servlets. `@Controller` classes are Servlets with annotation-based URL mapping. `@RequestParam` is `getParameter()`. You already know the concepts — Spring just removes the boilerplate.

Servlet/JSP Way (Manual)

Register Servlet in `web.xml`. Override `doGet()` / `doPost()`. Call `getParameter()` by name. Call `setAttribute()` then `forward()`. Write DAO with manual JDBC. One Servlet class per URL.

Spring MVC Way (Automated)

`@Controller` + `@RequestMapping` — no `web.xml` needed. `@GetMapping` / `@PostMapping` on methods. `@RequestParam` auto-binds form fields. Return model + view name — Spring handles forward. `@Repository` + JPA — no manual JDBC. One class handles many URLs.

PRO TIP — Servlet/JSP Knowledge Is Still Relevant

Spring Boot apps embedded in Tomcat still use the Servlet API under the hood. Every `@RestController` is ultimately a Servlet. Understanding the Servlet lifecycle helps you debug Spring Security filter chains, write high-performance middleware (Filters, Interceptors), and understand framework internals.

TRY THIS LIVE — Module Completion Checklist

Before the mini-project, verify all five flows work: (1) Login form -> `LoginServlet` -> session -> dashboard. (2) Register form -> validation -> DAO -> MySQL INSERT. (3) List students -> DAO -> JSP with JSTL `forEach`. (4) Logout -> `session.invalidate()` -> redirect to login. (5) Global `error.jsp` configured in `web.xml` for 404 and exceptions. All five working = you are ready for Campus Connect.

CASE STUDY — Infosys iCampus Portal Migration

Context — Infosys built iCampus, an internal training and campus portal for its 3,00,000+ employees, originally using Java Servlet and JSP technology.

Challenge — The original Servlet/JSP application became difficult to scale as new features (mobile, API access, third-party integrations) were added. But throwing away existing team knowledge was not an option.

What they did — Infosys migrated iCampus from raw Servlets to Spring MVC (and later Spring Boot),

retaining the same MVC architecture. Servlets became `@Controller` classes. JSPs were replaced with Thymeleaf. DAOs became `@Repository`. The concepts were identical — only the syntax changed.

Result — Migration was completed by teams who already knew Servlet/JSP. No retraining needed — they recognised every Spring concept from their foundations. The new system handles 10× the load with 40% less code.

Takeaway — What you built in this module is not legacy knowledge — it is the foundation every Java web developer carries into Spring and Spring Boot.

Source: Infosys Engineering Practices, 2022

“Every Spring Boot `@Controller` you write for the rest of your career is just a Servlet with a fancy hat. Know the hat. Know the head underneath it.”

WHERE TO GO NEXT

You have completed the Servlet and JSP module — you are a full-cycle Java web developer.

Explore deeper — Compare your manual Servlet code with the equivalent Spring Boot controller:
<https://spring.io/guides/gs/serving-web-content/>

Try on your own — Take your student registration project and convert one Servlet to a Spring Boot `@Controller` using `@GetMapping` and `@PostMapping`. Run both side-by-side and observe how the browser reaches both in the same way.

Connect the dots — Module 4 introduces Hibernate — replacing your manual JDBC DAO code with ORM annotations. Your Student POJO becomes a `@Entity` managed by a persistent context.

Going further — Read the Servlet 3.0 specification introduction (JSR-315) to understand where `@WebServlet`, async support, and multipart file upload were standardised.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java web development mentor. Summarise the journey from raw Servlets to Spring MVC in simple Hinglish. Use the analogy of going from making roti by hand to using a roti maker — explain what stays the same and what becomes easier. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 3

Interview Questions

◆ Web Technologies In Java

Rehearse these questions out loud, aiming for a 60-second answer each. The hint is only a nudge — try answering before you look at it. Compare your answer to the model answer afterward, not before.

1

What is the difference between a static and a dynamic web page?

Think about how the content is generated and whether it changes based on user input or server-side logic.

MODEL ANSWER

A static web page is a pre-written HTML file served to every user exactly as stored — the content never changes unless the developer edits the file manually. A dynamic web page generates its content on the fly, on the server, based on user input, database data, or session state. For example, a user's profile page shows their own data — that is dynamic. Java Servlets and JSP are technologies used to generate dynamic web content on the server side.

2

Explain the client-server model in the context of web applications.**MODEL ANSWER**

In the client-server model, the client (typically a browser) sends a request to the server over the network; the server processes the request and sends back a response. The client renders the HTML/CSS/JS to display the page. In Java web applications, Apache Tomcat is the server, and Servlets run inside it to handle requests. The browser never directly accesses the database or server files — all that happens on the server side. This separation keeps the business logic and data secure while the client only receives what it needs to display.

3

How does HTTP communication work between a browser and a web server?**MODEL ANSWER**

HTTP (HyperText Transfer Protocol) is a request-response protocol. The browser sends an HTTP request containing: the method (GET, POST, etc.), the URL, headers (like Accept or Cookie), and optionally a body (for POST). The server processes the request and returns an HTTP response containing: a status code (200 OK, 404 Not Found, etc.), response headers, and the body (HTML, JSON, or file data). HTTP is stateless — each request is independent, which is why mechanisms like cookies and sessions are needed to maintain user state.

4

In a typical Java web application, describe the data flow from a user submitting a form on the frontend to the database storing the data.

Consider the roles of the frontend, servlet, and database.

MODEL ANSWER

1) User fills a form and clicks submit — the browser sends an HTTP POST request with form data. 2) Tomcat receives the request and routes it to the mapped Servlet. 3) The Servlet reads form parameters using `request.getParameter()`. 4) The Servlet validates the input and calls a service or DAO class. 5) The DAO uses JDBC to execute an INSERT statement to store the data in MySQL. 6) If successful, the Servlet either sends a response directly or forwards/redirects to a JSP confirmation page. 7) The browser displays the result.

5

What is a Java Servlet and why is it used in web development?

MODEL ANSWER

A Java Servlet is a Java class that runs on a web server (like Tomcat) and handles HTTP requests and generates HTTP responses. It extends `HttpServlet` and overrides methods like `doGet()` and `doPost()` to handle different types of requests. Servlets are used to build dynamic web applications — they process form submissions, retrieve data from databases, manage sessions, and generate responses (HTML or JSON). They are the backbone of Java EE web development, providing a standardized way to handle web requests server-side.

6

List and briefly describe the three main methods in the Servlet lifecycle.

MODEL ANSWER

`init()`: called once when the Servlet is first loaded by the container — used to initialize resources like database connections. `service()`: called for every HTTP request — it routes the request to `doGet()` or `doPost()` based on the HTTP method. `destroy()`: called once when the Servlet is taken out of service (e.g., server shutdown) — used for cleanup like closing connections. The container manages this lifecycle; the developer overrides `init()` and `destroy()` for resource management, and `doGet()/doPost()` for request handling.

7

What is the purpose of the web.xml deployment descriptor in a Java web application?

Think about configuration and how servlets are mapped.

MODEL ANSWER

`web.xml` is the configuration file for a Java web application. It declares which Servlet classes exist, maps URLs to specific Servlets (so `/login` maps to `LoginServlet`), sets initialization parameters, defines welcome files, configures error pages, sets up filters and listeners, and controls security constraints. With Servlet 3.0+, annotations like `@WebServlet` can replace much of `web.xml`, but `web.xml` remains useful for application-wide configuration and for settings that cannot be expressed with annotations.

8

How would you deploy and access a simple 'Hello Servlet' on Apache Tomcat?

MODEL ANSWER

Write a class that extends `HttpServlet` and overrides `doGet()`. Annotate it with `@WebServlet('/hello')`. Package the project as a WAR file. Copy the WAR to Tomcat's `webapps/` directory or use your IDE's Tomcat integration to deploy. Start Tomcat. Access it in a browser at `http://localhost:8080/YourAppName/hello`. Tomcat deploys the WAR automatically when it starts. The Servlet container instantiates the Servlet class and calls `doGet()` when a GET request arrives at `/hello`.

9

What is the difference between the `doGet()` and `doPost()` methods in an `HttpServlet`?

Consider when each method is typically used.

MODEL ANSWER

`doGet()` handles HTTP GET requests — typically used for reading data or navigating to a page. Parameters are visible in the URL query string, making GET requests bookmarkable but unsuitable for sensitive data. `doPost()` handles HTTP POST requests — used for submitting forms, sending data that modifies server state, or handling sensitive information (passwords, payment details). POST data is in the request body, not the URL. Servlets override both as needed; a login form should always use POST. GET requests can be cached and bookmarked; POST requests should not.

10

How do you retrieve form parameters sent from a client to a servlet?

Recall the method from the `HttpServletRequest` object.

MODEL ANSWER

Use `request.getParameter('parameterName')` where `request` is the `HttpServletRequest` passed into `doGet()` or `doPost()`. It returns the value as a `String`, or `null` if the parameter is not present. For multiple values with the same name (e.g., checkboxes), use `request.getParameterValues('name')` which returns a `String[]`. For reading request body data directly (e.g., JSON), use `request.getReader()` or `request.getInputStream()`. Always validate and sanitize the retrieved values before using them.

11

Write a simple code snippet in a servlet that reads a 'username' parameter from a request and prints it in the response.

MODEL ANSWER

```
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException,
IOException { String username = request.getParameter('username'); response.setContentType('text/html');
PrintWriter out = response.getWriter(); if (username != null && !username.isEmpty()) {
out.println('<h2>Welcome, ' + username + '!'</h2>'); } else { out.println('<p>Username not provided.</p>'); } }
— response.getWriter() returns a PrintWriter to write HTML directly to the response. Always validate
parameters before outputting them.
```

12

Explain the difference between ServletConfig and ServletContext.

One is specific to a servlet, the other is shared across the application.

MODEL ANSWER

ServletConfig holds configuration information specific to a single servlet — initialization parameters defined in web.xml for that servlet, accessible via getInitParameter(). Each servlet has its own ServletConfig.

ServletContext represents the entire web application and is shared by all servlets — it holds application-wide parameters, resources, and allows servlets to share data and communicate with each other. Access ServletContext via getServletContext(). Use ServletConfig for per-servlet settings (like a template path) and ServletContext for app-wide settings (like a database URL).

13

How would you store and access a database connection string as a context parameter in your web application?

MODEL ANSWER

In web.xml, define:

```
<context-param><param-name>db.url</param-name><param-value>jdbc:mysql://localhost/mydb</param-value></context-param>
```

— In any Servlet, access it with: String dbUrl = getServletContext().getInitParameter('db.url'); — This centralizes configuration so changing the database URL only requires editing web.xml, not every servlet or DAO class. For more advanced applications, externalize this into a .properties file or environment variable, but context parameters are a clean starting point.

14

What is the purpose of sendRedirect() in a servlet? How does it differ from RequestDispatcher.forward()?

Consider what happens to the client's browser and the URL.

MODEL ANSWER

sendRedirect() tells the browser to make a new request to a different URL — the browser URL changes and a new HTTP request is made. This is an external redirect and works across different web applications.

RequestDispatcher.forward() transfers control internally on the server — the browser URL does not change, the original request and response objects are passed to the target resource, and no new HTTP request is made. Use forward() to share request data between a servlet and JSP. Use sendRedirect() after form submission (Post-Redirect-Get pattern) to prevent form resubmission on browser refresh.

15

Name and briefly describe the meaning of HTTP status codes 200, 302, and 404.

MODEL ANSWER

200 OK: the request was successful — the server processed it and is returning the requested content. 302 Found: the resource has temporarily moved to a different URL — the browser should redirect to the Location header's URL. This is what sendRedirect() generates. 404 Not Found: the requested resource does not exist on the server — the URL is wrong or the resource has been deleted. Other commonly asked codes: 500 (Internal Server Error — the server crashed processing the request) and 401 (Unauthorized — authentication required).

16

After a successful login, how would you redirect a user from a `LoginServlet` to a `DashboardServlet`?

MODEL ANSWER

After verifying credentials and creating the session: `response.sendRedirect(request.getContextPath() + '/dashboard');` — `getContextPath()` returns the application's context path, ensuring the URL is correct regardless of deployment. The browser makes a new GET request to `/dashboard`. The `DashboardServlet` checks if a valid session exists; if not, it redirects back to the login page. This follows the Post-Redirect-Get pattern, preventing the login form from being resubmitted when the user refreshes the dashboard page.

17

What are cookies, and how are they used for session tracking in Java web applications?

MODEL ANSWER

Cookies are small pieces of data stored on the client's browser and sent with every subsequent request to the same server. A web application creates a cookie with: `Cookie c = new Cookie('user', 'priya');` `response.addCookie(c);` — On the next request, the browser sends it back, and the server reads it with `request.getCookies()`. For session tracking, a session ID cookie is created when the session starts, allowing the server to identify the user across requests. Cookies can have expiry times, domain restrictions, and security flags (`HttpOnly`, `Secure`).

18

Explain how the `HttpSession` API helps maintain user login state across multiple requests.

MODEL ANSWER

HTTP is stateless, so the server has no memory of previous requests. `HttpSession` solves this: when a user logs in, you create or retrieve a session with `request.getSession()`, store user data in it: `session.setAttribute('user', userObject)`, and the server generates a unique session ID stored in a cookie (`JSESSIONID`). On every subsequent request, the browser sends this cookie, and the server retrieves the same session object with `request.getSession(false)` and reads the stored attributes. The session expires after inactivity or when `session.invalidate()` is called.

19

Describe one scenario where URL rewriting would be used for session tracking.

MODEL ANSWER

URL rewriting is used when the client has cookies disabled — the session ID is appended to every URL as a query parameter instead: `/dashboard?jsessionid=ABC123`. In Java, `response.encodeURL('/dashboard')` automatically appends the session ID if cookies are unavailable. Scenario: a corporate user's browser policy blocks all cookies. The application falls back to URL rewriting so sessions still work. Drawback: session IDs in URLs are visible (security risk) and URLs cannot be bookmarked meaningfully. It is used as a fallback, not preferred.

20

How would you keep a user logged in until they explicitly log out or the session expires? Briefly outline the steps.

MODEL ANSWER

On login: verify credentials, then `HttpSession session = request.getSession(); session.setAttribute('loggedInUser', user);` Set a session timeout in `web.xml`: `<session-timeout>30</session-timeout>` (minutes). On every protected page, check: `HttpSession s = request.getSession(false); if (s == null || s.getAttribute('loggedInUser') == null) { response.sendRedirect('/login'); return; }` On logout: `request.getSession().invalidate(); response.sendRedirect('/login');` — This pattern protects all pages and auto-expires idle sessions.

21

What is JSP and how does it differ from a servlet?

Think about who writes each, and how dynamic content is generated.

MODEL ANSWER

JSP (JavaServer Pages) allows developers to embed Java code inside HTML using special tags (`<% %>`, `<%= %>`), making it easier for web designers to create dynamic pages without writing pure Java. A Servlet is a pure Java class that generates HTML programmatically using `out.println()` — verbose and hard to design visually. JSP is compiled into a Servlet by the container at first access. The best practice is to use Servlets for logic and JSP for display (MVC), combining the strengths of both: clean HTML templating in JSP and clean business logic in Java.

22

Describe the JSP lifecycle in brief.

MODEL ANSWER

When a JSP is first requested: 1) Translation — the JSP container converts the .jsp file into a Java Servlet source file. 2) Compilation — the Servlet is compiled into bytecode. 3) Loading and initialization — the Servlet is loaded, and `jspInit()` is called. 4) Execution — for each request, `jspService()` is called, generating the response. 5) Destruction — when the server shuts down or the JSP is updated, `jspDestroy()` is called. Subsequent requests skip translation and compilation and go directly to execution, making later requests fast.

23

What is the purpose of JSP directives such as `<%@ page %>`?

MODEL ANSWER

JSP directives provide global settings for the entire JSP page, processed at translation time. `<%@ page %>` sets page-level attributes like language, content type, import statements, and error page: `<%@ page contentType='text/html' import='java.util.*' errorPage='error.jsp' %>`. `<%@ include %>` statically includes another file's content at translation time — useful for headers and footers. `<%@ taglib %>` declares a tag library (like JSTL), giving access to custom tags. Directives apply to the whole page, unlike scriptlets which are inline code.

24

How would you use Expression Language (EL) in JSP to display a user's name stored in a request attribute?

MODEL ANSWER

In the Servlet: `request.setAttribute('userName', 'Priya');` then `RequestDispatcher.forward()` to the JSP. In the JSP, use EL: `<h2>Welcome, ${userName}!</h2>` — EL (Expression Language) automatically accesses scoped attributes (request, session, application) using `${attributeName}`. It is cleaner than scriptlets (`<%= %>`) because it keeps JSP focused on display without Java code. EL also accesses bean properties: `${user.name}` calls `getUser().getName()`. EL is null-safe — it silently renders nothing for null values.

25

Compare dynamic rendering in JSP with dynamic rendering in a frontend framework like React.

MODEL ANSWER

In JSP, dynamic content is generated on the server — the server combines data with the template and sends complete HTML to the browser. The browser receives ready-to-display HTML; JavaScript is optional. In React, the browser receives a mostly empty HTML page and a JavaScript bundle. React runs client-side, fetches data from APIs, and dynamically builds and updates the DOM. JSP is server-side rendering (SSR), React is typically client-side rendering (CSR). SSR is simpler and SEO-friendly; CSR enables richer interactivity without full page reloads.

26

What does the `<jsp:include>` action tag do in a JSP page?

Think about code reuse and including common layouts.

MODEL ANSWER

`<jsp:include page='header.jsp' />` dynamically includes another JSP file at request time, each time the page is requested. Unlike `<%@ include %>` (static, at translation time), `<jsp:include>` processes the included file independently and merges its output into the current page. This is useful for dynamic content like a navigation bar that shows the logged-in user's name — the included file can access the same request/session. It promotes reuse by sharing common UI components across pages without duplicating markup.

27

How would you use `<jsp:useBean>` and `<jsp:setProperty>` to work with JavaBeans in a JSP?

MODEL ANSWER

`<jsp:useBean id='student' class='com.tops.Student' scope='request' />` — retrieves an existing bean or creates a new one in the specified scope. `<jsp:setProperty name='student' property='*' />` — automatically maps request parameters to bean properties with matching names, calling the appropriate setters. `<jsp:getProperty name='student' property='name' />` — reads and outputs a property. JavaBeans used with JSP must have a no-arg constructor and standard getters/setters. This approach reduces scriptlet code in JSP.

28

Explain the benefit of using custom tag libraries in JSP.

MODEL ANSWER

Custom tag libraries (like JSTL — JSP Standard Tag Library) let you replace Java scriptlet code in JSP with HTML-like tags. For example, instead of a Java for loop in a scriptlet, JSTL offers `<c:forEach items='${list}' var='item'>${item.name}</c:forEach>`. This keeps JSP files clean and readable for HTML designers who may not know Java. It separates concerns, reduces the risk of bugs from mixing Java and HTML, and makes templates easier to maintain. JSTL provides tags for iteration, conditions, formatting, and XML/SQL.

29

If you want to include a common header and footer in all your JSP pages, how would you achieve this?

MODEL ANSWER

Create separate JSP files: header.jsp and footer.jsp. In each main page, include them: `<%@ include file='header.jsp' %>` at the top and `<%@ include file='footer.jsp' %>` at the bottom for static inclusion (compiled together), or `<jsp:include page='header.jsp' />` for dynamic inclusion (processed at runtime). Static include is more efficient since it is done at compile time. Dynamic include is needed if the header shows user-specific data. This approach ensures consistent UI across all pages and a single place to update the header/footer.

30

Explain the Model-View-Controller (MVC) pattern as it applies to Java web applications using Servlets and JSP.

Describe the roles of Servlet, JSP, and JavaBean.

MODEL ANSWER

MVC separates concerns into three layers. Model: Java classes (like Student, Event) and DAO classes that interact with the database — they represent the data and business logic. View: JSP pages that display data to the user — they should contain minimal or no Java code. Controller: Servlets that receive HTTP requests, call the Model, and decide which View to show. A request comes to the Servlet (Controller), it calls the DAO (Model), stores results as request attributes, then forwards to a JSP (View) for display. This makes the app modular, testable, and maintainable.

31

In a student registration application, which component would handle data validation and which would display the registration form?

MODEL ANSWER

The Servlet (Controller) handles data validation — it reads submitted form parameters, checks for required fields, validates formats (email, phone), and ensures no duplicates exist in the database. If validation passes, it calls the DAO to save the record. If it fails, it sets error messages as request attributes and forwards back to the JSP. The JSP (View) both displays the registration form initially and re-displays it with error messages after a failed submission, using EL to show the error messages set by the Servlet.

32

How does data flow from a form submitted by the user to being displayed on a JSP page using the MVC pattern?

MODEL ANSWER

1) User submits the HTML form — browser sends HTTP POST to a URL. 2) Tomcat routes the request to the mapped Servlet. 3) Servlet reads parameters with `request.getParameter()`. 4) Servlet calls a service/DAO method with validated data. 5) DAO queries the database and returns a result (e.g., a list of records). 6) Servlet stores the result: `request.setAttribute('students', studentList)`. 7) Servlet forwards to a JSP: `request.getRequestDispatcher('/students.jsp').forward(request, response)`. 8) JSP reads the attribute via EL: `${students}` and renders it in HTML.

33

Why is server-side validation important, even if you have client-side validation in your forms?

MODEL ANSWER

Client-side validation (JavaScript) runs in the browser and can be bypassed — a user can disable JavaScript, use browser developer tools to remove validation attributes, or send HTTP requests directly using tools like Postman or curl. Server-side validation in the Servlet is the security layer that cannot be bypassed. It ensures that invalid, malicious, or malformed data is never saved to the database regardless of how the request was made. Client-side validation improves user experience with instant feedback; server-side validation ensures data integrity and security.

34

Write a simple server-side check in a servlet to ensure a 'password' parameter is not empty.

MODEL ANSWER

```
String password = request.getParameter('password'); if (password == null || password.trim().isEmpty()) {
    request.setAttribute('error', 'Password is required.');
```

`request.getRequestDispatcher('/register.jsp').forward(request, response); return; }` — Check for null (parameter not present) and empty string (user submitted blank). Use `trim()` to also reject whitespace-only passwords. Set an error attribute and forward back to the form JSP so the user can correct it. The return statement is critical to stop further processing after the forward.

35

How would you configure a custom error page in JSP to handle 404 errors?

MODEL ANSWER

In `web.xml`: `<error-page><error-code>404</error-code><location>/error404.jsp</location></error-page>` — Create `error404.jsp` with a user-friendly 'Page Not Found' message and a link back to the home page. Similarly configure 500 for server errors. In the error JSP, you can access exception details with `${pageContext.exception}`. This replaces the default browser or Tomcat error page with a branded, user-friendly page. You can also configure error pages by exception type: `<exception-type>java.lang.NullPointerException</exception-type>`.

36

Describe how you would display an 'Invalid Credentials' message to a user after a failed login attempt.

MODEL ANSWER

In the LoginServlet, if credentials don't match: `request.setAttribute('loginError', 'Invalid username or password.');` `request.getRequestDispatcher('/login.jsp').forward(request, response);` — In `login.jsp`: `<c:if test='${not empty loginError}'><p class='error'>${loginError}</p></c:if>` — Using `forward()` rather than `redirect()` preserves the request attributes. The JSTL `<c:if>` tag conditionally displays the message only when it is set. Avoid revealing whether it was the username or password that was wrong — this prevents username enumeration attacks.

37

What steps are involved in connecting a Java Servlet to a MySQL database using JDBC?

MODEL ANSWER

1) Add MySQL Connector/J JAR to the project's WEB-INF/lib folder. 2) Store connection details (URL, user, password) as context parameters in `web.xml` or a config file. 3) Create a `DBUtil/ConnectionManager` class that provides a `Connection` object using `DriverManager.getConnection()`. 4) In the DAO class, call `DBUtil.getConnection()` before each operation (or use a single connection per session). 5) Use `PreparedStatement` for all queries. 6) Close all resources in finally blocks or try-with-resources. 7) Test by running a simple `SELECT` query from a Servlet.

38

Write a code snippet to fetch a list of students from a database and forward it to a JSP for display.

MODEL ANSWER

In the Servlet: `List<Student> students = studentDAO.getAllStudents(); request.setAttribute('students', students); request.getRequestDispatcher('/students.jsp').forward(request, response);` — In `studentDAO.getAllStudents()`: execute `SELECT * FROM students`, create a `Student` object for each row, add to a list, return it. In `students.jsp`: `<c:forEach items='${students}' var='s'><tr><td>${s.name}</td><td>${s.email}</td></tr></c:forEach>` — This follows the MVC pattern: DAO (Model), JSP (View), Servlet (Controller).

39

How can you display database records in a table format on a JSP page?

MODEL ANSWER

`<table border='1'><tr><th>Name</th><th>Email</th><th>Marks</th></tr><c:forEach items='${students}' var='student'><tr><td>${student.name}</td><td>${student.email}</td><td>${student.marks}</td></tr></c:forEach></table>` — The Servlet sets the list as a request attribute, and the JSP iterates over it with JSTL's `<c:forEach>`. Each row's data is accessed via EL property notation `${student.name}` which calls `getName()` on the `Student` object. Add CSS classes or Bootstrap for better styling.

40

Explain how the MVC pattern helps when integrating JDBC with Servlets and JSP for database operations.

MODEL ANSWER

Without MVC, database code, business logic, and HTML rendering would be mixed in one class or file — making it unreadable and unmaintainable. MVC separates them: the DAO (part of Model) contains all JDBC code; the Servlet (Controller) orchestrates the flow — calling the DAO, handling errors, setting attributes; the JSP (View) only renders the data passed to it, with no knowledge of JDBC. This means a developer can change the database query without touching the JSP, and a designer can update the HTML without touching the Java logic.

41

If you were to build a small web portal for student registration and event posting, what would be the main components and their responsibilities?

Think about controllers, models, views, and how data flows.

MODEL ANSWER

Models/DAOs: Student.java and Event.java POJOs; StudentDAO and EventDAO for JDBC operations. Controllers/Servlets: RegisterStudentServlet (handles POST from registration form), EventListServlet (fetches and forwards events), LoginServlet, LogoutServlet. Views/JSPs: register.jsp, login.jsp, events.jsp, eventDetail.jsp, home.jsp. Utilities: DBUtil for connection management. The web.xml maps URLs to servlets. A session filter protects pages that require login. Each component has a single responsibility, making the codebase maintainable.

42

Describe the end-to-end flow of a user registering for an event in the Campus Connect Portal project.

MODEL ANSWER

1) User is logged in (session has 'user' attribute). 2) User views events.jsp listing available events. 3) User clicks 'Register' for an event — sends POST to /registerEvent. 4) EventRegistrationServlet reads userId from session and eventId from request parameter. 5) Validates that the user isn't already registered. 6) RegistrationDAO inserts a row into the event_registrations table. 7) On success, Servlet sets a success message attribute and redirects to events.jsp. 8) events.jsp displays the success message and the updated registration status.

43

How could you use an AI tool like ChatGPT or GitHub Copilot to help you debug a servlet or JSP issue, and how would you make sure the suggestions are safe to use?

Think about using AI for code help but also the need to test and review suggestions.

MODEL ANSWER

Paste the error stack trace and relevant servlet/JSP code into ChatGPT and ask for an explanation of the error. For a NullPointerException, it will typically identify which variable is null and why. For a 500 error, it helps trace the Tomcat log. Before applying the fix, understand why the change works — don't copy blindly. Verify that suggested code still uses PreparedStatement (not Statement) for any SQL. Test the fix with multiple scenarios including edge cases. Check that session handling and redirects still work correctly. Review any security implications of changes.

44

How might AI-powered tools speed up your web development process in Java, and where would you still rely on your own skills?

MODEL ANSWER

AI tools speed up boilerplate generation — servlet templates, DAO CRUD methods, web.xml snippets, JSP forms with JSTL. They help with syntax lookups, explain error messages quickly, and suggest design patterns. Where you still need your own skills: understanding the full request-response lifecycle to debug complex issues; designing the application architecture and MVC structure; making security decisions (session management, SQL injection prevention); understanding why code works, not just that it works; and adapting AI suggestions to your specific schema, business rules, and project conventions.